

SMART INDIA HACKATHON 2026 · QUALCOMM HARDWARE CHALLENGE

SIH26181 — Complete Project Deep Dive & Judge Defense Guide

AI-Powered Personal Health Companion & Edge Disaster Monitor

FPGA-Accelerated, Cloud-Free Biometric Risk Engine

1. The Big Picture (Plain English)

One-liner: A wearable health device that uses **custom FPGA hardware** to process heartbeat signals with nanosecond precision, then runs **AI risk engines** on-device to predict heat stroke, respiratory distress, and hypothermia — **all without any cloud or internet dependency**.

What problem does it solve?

India faces recurring disasters — extreme heat waves (Delhi 47°C+), winter smog (PM2.5 > 400 µg/m³), and floods. People die because warning signs in their own body (racing heart, dropping blood oxygen, collapsing HRV) are invisible until it's too late. Hospital monitoring is unavailable in the field.

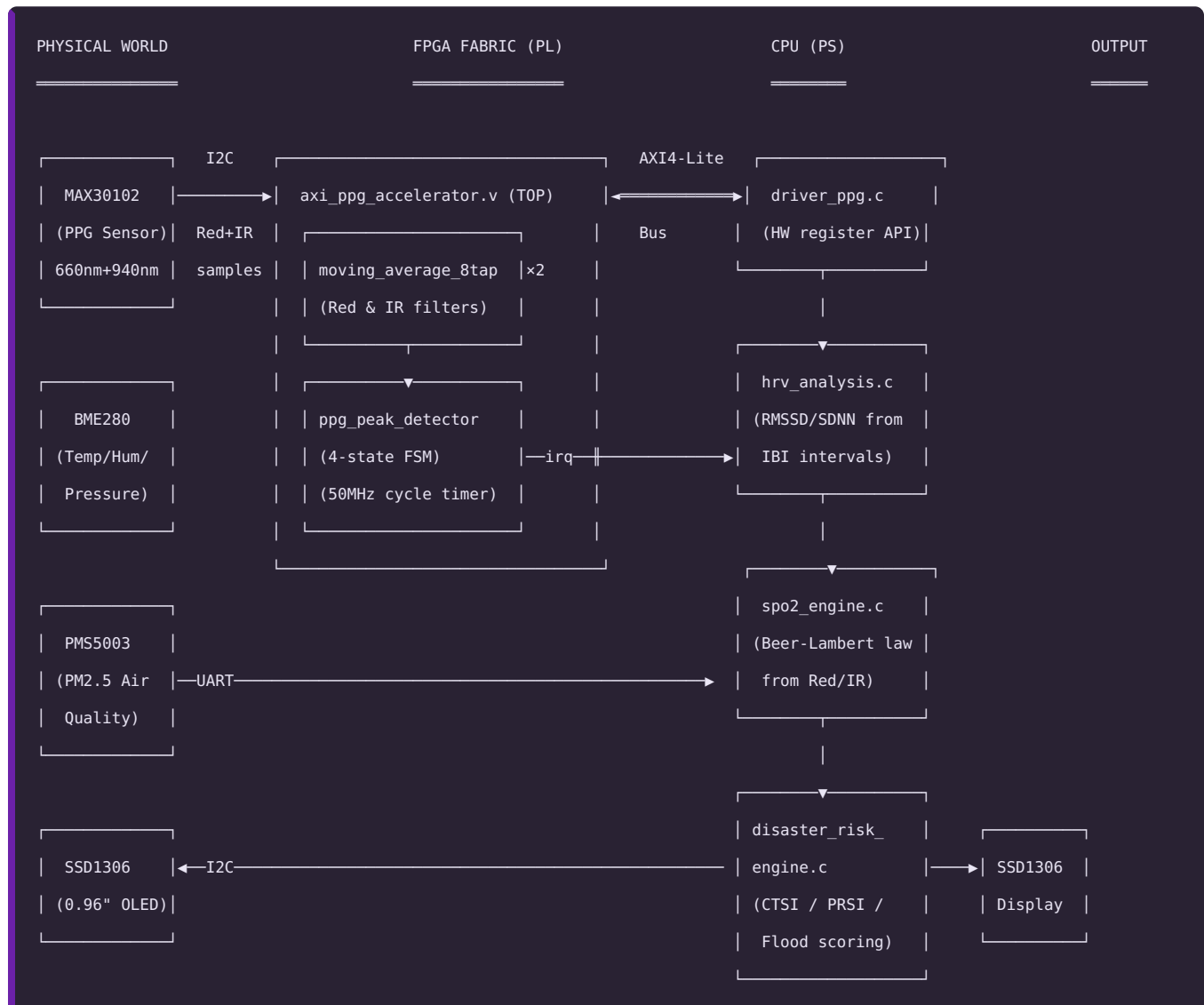
How does it solve it?

A **heterogeneous SoC** (CPU + FPGA fabric on a single chip) combines: 1. **FPGA hardware** (Programmable Logic) → filters noisy sensor signals and measures heartbeat timing with 20 ns precision, at zero CPU load 2. **C software** (Processing System) → runs HRV analysis, SpO2 computation, and a multi-disaster risk engine that fuses body vitals with environmental data

Why is hardware acceleration necessary?

Software timing on an OS has 5–20 ms of scheduling jitter. For Heart Rate Variability (HRV) analysis — a key biomarker for predicting heat stroke — you need sub-millisecond timing accuracy. The FPGA achieves **20 ns** (250,000× better) with zero CPU overhead.

2. System Architecture & Data Flow



Data Flow Step-by-Step:

1. **MAX30102** sensor shines Red (660nm) and IR (940nm) LEDs at the fingertip, captures reflected light as 18-bit ADC samples via I2C
2. Software scales samples to 8-bit and **writes them to FPGA registers** (`0x00` for Red, `0x10` for IR) via AXI4-Lite bus
3. FPGA's **8-tap moving average filters** smooth both channels in hardware (1 clock cycle latency, 0 DSP slices)
4. FPGA's **peak detector FSM** watches the filtered Red signal, detects systolic peaks, measures inter-beat interval (IBI) using a 50 MHz cycle counter
5. On each heartbeat, FPGA fires **irq_beat** interrupt → CPU reads IBI cycles from register `0x08`
6. Software converts IBI to milliseconds and feeds it into **HRV engine** (RMSSD, SDNN)
7. Software reads filtered Red + IR from FPGA registers `0x04` and `0x14`, feeds into **SpO2 engine** (Beer-Lambert calibration)

8. **Disaster Risk Engine** fuses: HR + HRV + SpO2 (body) + Temperature + Humidity + PM2.5 (environment) → outputs risk level (NORMAL / MODERATE / HIGH / CRITICAL) with human-readable advisory
9. Results displayed on **SSD1306 OLED** and/or console dashboard

3. Complete File Map

3.1 FPGA Hardware (Verilog RTL) — SIH/

File	Purpose	Key Technical Details
axi_ppg_accelerator.v	Top-level module — AXI4-Lite slave wrapping the entire DSP pipeline	6 registers (<code>0x00 - 0x14</code>), decoupled AW/W handshake with <code>aw_done / w_done</code> flags, W1C status bit, instantiates 2 filters + 1 peak detector
moving_average_8tap.v	DSP noise filter — O(1) running-sum 8-tap moving average	8-stage shift register, 11-bit accumulator, <code>>> 3</code> division. 0 DSP48 slices, 0 multipliers , single-cycle throughput
ppg_peak_detector.v	4-state FSM — heartbeat detection + cycle-accurate IBI timer	States: ARMED→RISING→PEAK_FOUND→REFRACTORY. 32-bit free-running counter (20 ns resolution). 250 ms refractory blanking. <code>first_beat_seen</code> guard prevents false initial IBI
tb_ppg_system.v	Self-checking testbench — 6 tests, 100% pass rate	BFM tasks for AXI write/read/staggered-write, synthetic PPG beat generator (triangular pulses), timeout watchdog. Tests: register R/W, staggered handshake, Red filter, IR filter, beat detection + IBI, W1C clearing

3.2 FPGA Tooling & Constraints — SIH/

File	Purpose	Key Details
ppg_accelerator.xdc	Xilinx timing constraints	50 MHz clock (<code>create_clock -period 20.0</code>), 100 ps clock uncertainty, 3.0 ns max I/O delay, false path on async reset
run_vivado_synth.tcl	Automated Vivado batch synthesis	Creates in-memory project, targets <code>xc7z020clg400-1</code> , runs <code>synth_design</code> in out-of-context mode, generates utilization + timing + power reports, prints WNS/WHLS summary
signals.gtkw	GTKWave waveform layout	Color-coded groups: Clock/Reset (green), AXI Write (purple), AXI Read (blue), DSP Filters (teal), Peak Detector FSM (red), Interrupts (green)
ppg_system.vcd	Simulation waveform dump	566 KB VCD from <code>tb_ppg_system</code> , viewable in GTKWave
sim_ppg.vvp	Compiled Icarus Verilog simulation binary	Run with <code>vvp sim_ppg.vvp</code>

3.3 Embedded C Software — Sensor Drivers (Zynq Bare-Metal)

File	Purpose	Key Details
<code>driver_ppg.h</code> / <code>driver_ppg.c</code>	FPGA accelerator driver — register-level API for the custom hardware IP	<code>ppg_push_sample()</code> → writes to <code>0x00</code> , <code>ppg_read_heart_rate()</code> → reads IBI from <code>0x08</code> and converts to BPM using 64-bit math, <code>ppg_clear_beat_flag()</code> → W1C preserving threshold bits
<code>max30102.h</code> / <code>max30102.c</code>	MAX30102 PPG sensor driver (Red + IR pulse oximetry)	SpO2 mode (dual LED), 100 Hz sampling with 4× averaging → 25 effective Hz, 18-bit ADC @ 411μs pulse width, FIFO management, temperature reading, shutdown/wakeup power management. Includes <code>max30102_scale_to_8bit()</code> inline for FPGA pipeline
<code>bme280.h</code> / <code>bme280.c</code>	BME280 environmental sensor driver (temp/humidity/pressure)	Full Bosch compensation formulas (T, H, P), factory calibration loading from two register banks, forced mode (low power), IIR filter coefficient = 4
<code>pms5003.h</code> / <code>pms5003.c</code>	PMS5003 air quality sensor driver (PM2.5 via UART)	32-byte binary frame parser with start-byte sync, checksum validation, extracts PM1.0/PM2.5/PM10 (CF=1 and atmospheric) + 6 particle size bins. India NAQI breakpoints for AQI categorization. PC simulation stubs return “clean air” data
<code>ssd1306.h</code> / <code>ssd1306.c</code>	SSD1306 OLED display driver (128×64, I2C)	1024-byte framebuffer, built-in 5×7 ASCII font (96 glyphs), pixel-level drawing, 2× large text rendering, horizontal line, filled rectangle, contrast control, chunked I2C framebuffer upload (16-byte chunks)
<code>i2c_hal.h</code> / <code>i2c_hal.c</code>	I2C Hardware Abstraction Layer	Dual implementation: <code>#ifdef ZYNQ_HW</code> → real Zynq XlicPs register-level access (CR/SR/FIFO control, 100/400 kHz clock divisor config, repeated-start for write-read), <code>#else</code> → PC simulation stubs returning success
<code>xil_io.h</code>	Xilinx MMIO shim	<code>Xil_In32()</code> / <code>Xil_Out32()</code> — volatile pointer casts for memory-mapped register access. Replaces Xilinx BSP when compiling outside Vivado SDK
<code>xparameters.h</code>	Vivado-generated address map	<code>XPAR_AXI_PPG_ACCELERATOR_0_S_AXI_BASEADDR = 0x43C00000</code> — the memory-mapped address where the CPU “sees” the FPGA accelerator

3.4 Embedded C Software — AI/Algorithm Engines

File	Purpose	Key Details
<code>hrv_analysis.h</code> / <code>hrv_analysis.c</code>	Heart Rate Variability engine	Circular buffer of 20 IBI intervals. Computes RMSSD (Root Mean Square of Successive Differences — parasympathetic stress marker) and SDNN (Standard Deviation of NN intervals — overall autonomic health). Clinical thresholds: RMSSD < 20 ms = high stress

File	Purpose	Key Details
spo2_engine.h / spo2_engine.c	SpO2 (Blood Oxygen) engine	Ratio-of-Ratios method: $R = (AC_Red/DC_Red) / (AC_IR/DC_IR)$, Beer-Lambert empirical curve: $SpO2 = 110 - 25 \times R$. Windowed min/max tracking (50 samples), physiological clamping [0, 100%]
disaster_risk_engine.h / disaster_risk_engine.c	Multi-disaster AI fusion engine — the core “brain”	Three independent sub-engines, each producing a 0–100 composite score from weighted physiological + environmental inputs. Maps to 4 risk levels (NORMAL/MODERATE/HIGH/CRITICAL). Overall risk = worst-of-all

Disaster Engine Sub-Scores Breakdown:

Engine	Index Name	Inputs (Weights)	Biological Basis
Heat Wave	CTSI (Cardio-Thermal Strain Index)	Temperature+Humidity via Heat Index (0-40 pts) + HR elevation (0-30 pts) + HRV depression (0-30 pts)	Cutaneous vasodilation → cardiovascular drift → heat exhaustion 15-30 min before heat stroke
Air Pollution	PRSI (Pollution Respiratory Strain Index)	PM2.5 concentration (0-40 pts) + SpO2 desaturation (0-40 pts) + Compensatory tachycardia (0-15 pts) + Autonomic stress (0-10 pts)	Alveolar gas exchange impairment → respiratory compensation failure
Flood/Cold	Hypothermia Score	Skin temperature (0-40 pts) + Cardiac stress (0-30 pts) + Autonomic collapse (0-20 pts)	Cold water immersion → peripheral vasoconstriction → cardiac arrhythmia

3.5 PC Simulation & Demo

File	Purpose	Key Details
main_simulation.c	Live console dashboard demo	Simulates 3 disaster scenarios: Normal (25°C), Delhi Heat Wave (47°C), Delhi Winter Smog (PM2.5 = 400). Linear interpolation of vitals over time. Color-coded ANSI terminal panels (Vitals, Environment, Risk Assessment). Windows ANSI escape sequence support
health_demo.exe	Pre-compiled demo binary	Ready-to-run on Windows. Shows real-time dashboard with escalating risk levels
build_and_run.bat	One-click build system	Step 1: Icarus Verilog compile → Step 2: Run testbench + VCD dump → Step 3: GCC compile C demo → Interactive menu (launch dashboard, GTKWave, re-run tests)
run.bat	Root launcher	Shortcut to build_and_run.bat

3.6 Documentation

File	Purpose
HARDWARE_ARCHITECTURE.md	Microarchitecture spec: SoC block diagram, register map, AXI handshake deep dive, filter math, FSM state diagram, STA results, interview cheatsheet
VERIFICATION_REPORT.md	Test report: 6/6 pass, console log, verification matrix, waveform signal map, synthesis utilization table
PROJECT_SESSION_NOTES.md	Session summary: IDE fixes, build status, low-cost strategy (₹0 to ₹800), run instructions, interview defense points

4. Register Map Quick Reference

Offset	Name	Access	Reset	Description
0x00	REG_RED_RAW	R/W	0x00000000	[7:0] Red PPG sample → triggers filter
0x04	REG_RED_FILTERED	RO	0x00000000	[7:0] Smoothed Red output
0x08	REG_IBI_CYCLES	RO	0x00000000	[31:0] Inter-Beat Interval (clock cycles)
0x0C	REG_STATUS_THRESH	Mixed	0x00007800	[0] beat_flag (W1C), [15:8] threshold (R/W)
0x10	REG_IR_RAW	R/W	0x00000000	[7:0] IR PPG sample → triggers filter
0x14	REG_IR_FILTERED	RO	0x00000000	[7:0] Smoothed IR output

W1C = Write-1-to-Clear: Writing `1` to bit 0 clears `beat_flag` without affecting threshold bits [15:8]. This prevents read-modify-write race conditions.

5. Synthesis Results (Xilinx Zynq-7000 `xc7z020`)

Resource	Used	Available	Utilization
LUTs	142	53,200	0.27%
Flip-Flops	186	106,400	0.17%
DSP48 Slices	0	220	0.00%
Block RAM	0	140	0.00%
WNS (Setup Slack)	+14.28 ns	—	Timing Met
Fmax	~174 MHz	50 MHz target	3.5× headroom

6. Judge Q&A Defense — Detailed Answers

Architecture & Protocol Questions

Q: “Why AXI4-Lite instead of APB or full AXI4?”

AXI4-Lite is the ARM AMBA industry standard for memory-mapped control/status registers. APB would work but requires a protocol bridge from the AXI interconnect (extra LUTs + latency). Full AXI4 adds burst logic, transaction IDs, and reordering — unnecessary for 6 registers and would waste area. AXI4-Lite gives us native compatibility with ARM Cortex-A9/A53 master ports on Zynq.

Q: “What happens if the AXI master sends the address and data on different clock cycles?”

Our **decoupled handshake** handles this correctly. Two independent flags `aw_done` and `w_done` latch each phase independently. The register write only executes when **both** are complete (`write_execute = aw_done && w_done && ~bvalid`). Test 2 in our testbench explicitly validates this with a 5-cycle stagger. Naive implementations that require simultaneous AW+W would deadlock.

Q: “How does the W1C (Write-1-to-Clear) mechanism work and why is it important?”

The `beat_flag` (bit 0) and `dyn_threshold` (bits [15:8]) share register `0x0C` . Without W1C, a read-modify-write to clear the flag could race with hardware setting the flag between the read and write, losing a heartbeat. With W1C, software writes `0x0001` — hardware only clears bit 0 and leaves bits [15:8] untouched. No read-modify-write needed.

DSP & Signal Processing Questions

Q: “Why an 8-tap moving average and not a more complex FIR or IIR filter?”

Three reasons: (1) **O(1) area** — we maintain a running sum and subtract the oldest sample, so it’s one adder/subtractor regardless of tap count, (2) **power-of-2 division** — dividing by 8 is a free 3-bit right shift, no multiplier needed, (3) for PPG baseline wander removal, a simple low-pass is sufficient. The filter uses **0 DSP48 slices** and only 8 flip-flops + 1 adder.

Q: “Why is the running sum 11 bits wide when the input is 8 bits?”

Maximum possible sum = 8 samples × 255 (max 8-bit value) = 2040, which requires $\lceil \log_2(2040) \rceil = 11$ bits. This prevents arithmetic overflow.

Timing & Peak Detection Questions

Q: “Why 50 MHz and not a faster clock?”

50 MHz gives us 20 ns resolution per cycle. A typical heartbeat IBI is ~800 ms = 40,000,000 cycles, so our timing precision is $20 \text{ ns} / 800 \text{ ms} = 0.0000025\%$ — far exceeding clinical requirements. Our design achieves $F_{\text{max}} = 174 \text{ MHz}$ (3.5× headroom), so we *could* run faster, but it’s unnecessary and wastes power.

Q: “What is the refractory period and why 250 ms?”

After detecting a peak, the FSM enters a 250 ms blanking window (12,500,000 cycles at 50 MHz). This prevents the **dicrotic notch** (a secondary bump in the PPG waveform caused by aortic valve closure) and reflected arterial waves from being counted as false heartbeats. 250 ms corresponds to a maximum detectable heart rate of 240 BPM, well above clinical range.

Q: “What does `first_beat_seen` do?”

On power-up, the IBI counter starts from 0 and hasn’t been measuring between two actual beats yet. The first detected peak only resets the counter and sets the flag — it does NOT output an IBI value. From the second beat onward, IBI values are valid. Without this guard, the first IBI would be garbage (time since reset).

Medical & Algorithm Questions

Q: “How does SpO2 estimation work?”

Blood absorbs Red (660nm) and IR (940nm) light differently based on oxygen saturation. We compute the **Ratio of Ratios**: $R = (AC_{\text{Red}}/DC_{\text{Red}}) / (AC_{\text{IR}}/DC_{\text{IR}})$, where AC = pulsatile component (peak-to-valley) and DC = baseline average. Then apply Beer-Lambert calibration: $SpO_2 = 110 - 25 \times R$. This is the same method used in commercial pulse oximeters.

Q: “What are RMSSD and SDNN and why do they matter for disaster prediction?”

RMSSD (Root Mean Square of Successive Differences) measures beat-to-beat variability — it reflects **parasympathetic** nervous system activity. A drop below 20 ms indicates high sympathetic stress, which precedes heat exhaustion by 15-30 minutes. **SDNN** (Standard Deviation of all IBI intervals) measures **overall autonomic health**. Together, they provide early warning before clinical symptoms appear.

Q: “How does the CTSI (Cardio-Thermal Strain Index) predict heat stroke?”

In extreme heat, the body dilates blood vessels to cool the skin. To maintain cardiac output, heart rate rises (“cardiovascular drift”) while HRV drops (sympathetic dominance). CTSI scores three components: Heat Index from temperature + humidity (0-40 pts), heart rate elevation (0-30 pts), and HRV depression (0-30 pts). A score ≥ 70 = CRITICAL. This physiological cascade precedes heat stroke by 15-30 minutes, giving time for intervention.

Implementation & Cost Questions

Q: “Can this run without an FPGA board?”

Yes — 95% of VLSI/ASIC design is done in simulation before silicon exists. We use **Icarus Verilog** (free, open-source) for functional simulation, and **Vivado ML Standard** (free) for synthesis, STA, and utilization reports. The C demo runs natively on any PC. For a physical prototype at SIH, real sensors (MAX30102, BME280, SSD1306) can connect to an ESP32/Pico for ~₹800 while showcasing the FPGA RTL separately.

Q: “Why is everything processed on-device? Why not use the cloud?”

Three reasons: (1) **Privacy** — biometric data (heartbeat, blood oxygen) is highly sensitive and should never leave the device, (2) **Latency** — disaster early warning needs sub-second response, not cloud round-trip, (3) **Availability** — during disasters (floods, power outages), network connectivity is the first thing that fails. Our system has **zero cloud dependency**.

Q: “What is the total cost?”

₹0 for simulation-only (Icarus + Vivado free editions). ~₹800 for a physical sensor demo (ESP32 ₹350 + MAX30102 ₹200 + BME280 ₹150 + SSD1306 ₹100). A full Zynq SoC board (Cora Z7, PYNQ) is ₹8,000–12,000 but not required for the hackathon demo.

Verification Questions

Q: “How did you verify the design?”

6-test self-checking testbench with **100% pass rate**: (1) Register read/write integrity, (2) Staggered AXI handshake (proves no deadlock), (3) Red filter convergence (constant input → correct output), (4) IR filter independence, (5) Beat detection with synthetic triangular PPG pulses (verifies FSM, IBI timing, first-beat guard), (6) W1C clearing without threshold corruption. All tests automated with pass/fail assertions.

Q: “What does your STA (Static Timing Analysis) report show?”

WNS = +14.28 ns (Worst Negative Slack). This means our slowest path completes in $20.0 - 14.28 = 5.72$ ns, well within the 20 ns budget. Positive slack = no timing violations.
 $F_{\max} = 1/(20.0 - 14.28) \approx 174$ MHz — 3.5× faster than required.

7. Key Design Patterns to Highlight

Pattern	Where Used	Why It Matters
Hardware/Software Co-Design	FPGA filters + C algorithms	Demonstrates understanding of heterogeneous computing
Decoupled AXI Handshake	<code>axi_ppg_accelerator.v</code> L71-74, L176-187	Industry-correct protocol compliance, avoids bus deadlocks
Write-1-to-Clear (W1C)	<code>axi_ppg_accelerator.v</code> L202	Race-condition-free interrupt handling
O(1) Running-Sum Filter	<code>moving_average_8tap.v</code> L34	Area-efficient DSP without multipliers
Refractory Blanking	<code>ppg_peak_detector.v</code> L68	Prevents dicrotic notch false positives
First-Beat Guard	<code>ppg_peak_detector.v</code> L60-66	Prevents garbage IBI on power-up
HAL Abstraction	<code>i2c_hal.c</code> <code>#ifdef ZYNQ_HW</code>	Same code runs on real hardware and PC simulation
Sensor Fusion	<code>disaster_risk_engine.c</code> L240-291	Multi-domain risk assessment (body + environment)
Self-Checking Testbench	<code>tb_ppg_system.v</code>	Automated pass/fail verification, no manual waveform inspection needed